

Documento Técnico de Arquitectura (DAS)

Sistema AFIP Mobile – Gestión Financiera Personal

Versión: 1.0.0

Fecha: Junio 2026

Tipo: Documento de Arquitectura de Software

Estándar de referencia: ISO/IEC/IEEE 42010 – Architecture Description

1. Introducción

1.1 Propósito

Este documento describe la arquitectura de software del sistema **AFIP Mobile**, incluyendo las decisiones de diseño, componentes, estructura, relaciones entre capas y diagramas técnicos. Sirve como referencia para el equipo de desarrollo, arquitectos y evaluadores técnicos.

1.2 Alcance

Cubre la arquitectura de los tres subsistemas:

- API REST** (Backend – Node.js)
- Frontend Web** (React)
- Aplicación Mobile** (Flutter)
- Capa de datos** (MySQL)

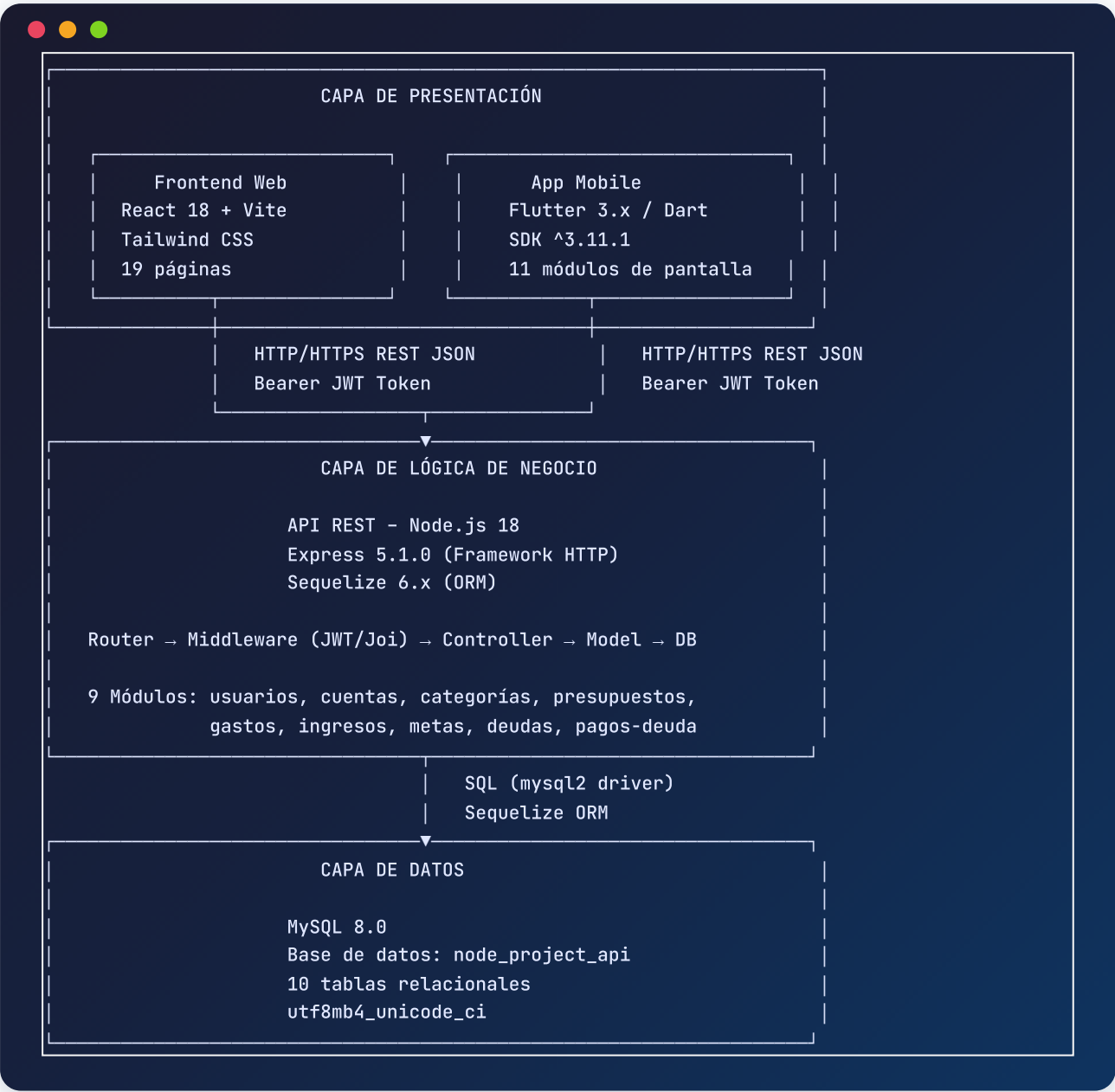
1.3 Visión del Sistema

AFIP Mobile es una plataforma de **gestión financiera personal** multi-frontend que sigue una arquitectura de **API-First**: una única API centralizada sirve tanto a la aplicación web como a la mobile.

2. Estilo Arquitectónico

2.1 Patrón Principal: API REST + Cliente/Servidor

El sistema sigue el estilo **REST (Representational State Transfer)** con una arquitectura **Cliente-Servidor de 3 capas**:



2.2 Patrón Interno de la API: MVC (Model-View-Controller)

La API implementa el patrón **MVC adaptado** para aplicaciones REST:

Capa	Componente	Responsabilidad
Router	<code>routers/*.router.js</code>	Definición de rutas HTTP y montaje de middlewares

Middleware	<code>middlewares/*.middleware.js</code>	Autenticación JWT y validación de esquemas
Controller	<code>controllers/*.controller.js</code>	Lógica de negocio y respuestas HTTP
Model	<code>models/*.model.js</code>	Definición de tablas y relaciones (Sequelize)
Schema	<code>schemas/*.schema.js</code>	Validación de datos de entrada (Joi)

3. Diagrama de Despliegue



4. Diagrama de Arquitectura de Componentes



| SQL via mysql2

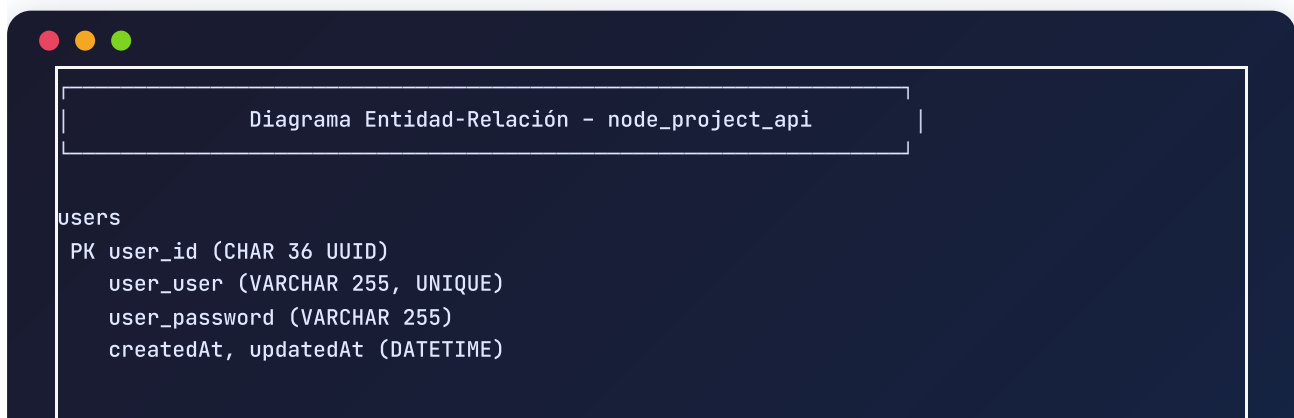
MySQL Database
node_project_api

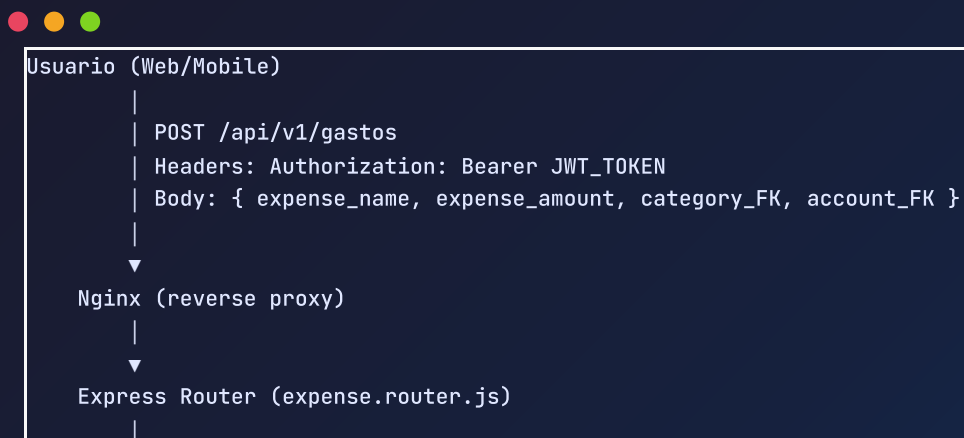
5. Diagrama de Clases





6. Diagrama de Base de Datos (ERD)





```

└─> verifyToken (auth.middleware.js)
    └─ Extrae token del header
    └─ jwt.verify(token, JWT_SECRET)
    └─ Si OK: inyecta req.user = { user_id, email }
    └─ Si FAIL: responde 401/403

└─> validateSchema (user.middleware.js)
    └─ Valida body.expense_name (required)
    └─ Valida body.expense_amount (number, min:1)
    └─ Valida body.category_FK (required)
    └─ Valida body.account_FK (required)
    └─ Si FAIL: responde 400 con detalles

└─> createExpense (expense.controller.js)
    └─ Busca Account por account_FK + user_FK
    └─ Verifica que saldo ≥ expense_amount
    └─ Crea registro en expenses (Sequelize)
    └─ Actualiza account_balance -= expense_amount
    └─ Responde: { ok: true, status: 200, body: expense }

```

8. Decisiones de Arquitectura

8.1 Por qué Node.js + Express 5

Factor	Justificación
Rendimiento	I/O no bloqueante; ideal para APIs REST con muchas peticiones concurrentes
Ecosistema	npm con librerías maduras (sequelize, jsonwebtoken, bcryptjs)
Familiaridad	Stack JavaScript unificado (backend = frontend)
Express 5	Mejor manejo de errores asíncronos sin wrapping manual

8.2 Por qué Sequelize ORM

Factor	Justificación
Abstracción	Evita SQL manual; queries parametrizadas previenen inyección SQL
Migraciones	Control de versiones del esquema de BD
Portabilidad	Cambio de dialecto sin reescribir código (MySQL → PostgreSQL)

Relaciones

`hasMany` , `belongsTo` simplifican joins

8.3 Por qué JWT para Autenticación

Factor	Justificación
Stateless	No requiere sesión en servidor; escalable horizontalmente
Estándar	Compatible con cualquier cliente HTTP (web, mobile, Postman)
Payload	Incluye <code>user_id</code> para filtrar datos por usuario en cada query

8.4 Por qué Flutter para Mobile

Factor	Justificación
Cross-platform	Un único código para Android, iOS, Web y Desktop
Rendimiento	Compilación AOT a código nativo
UI	Widgets propios (no depende del SO); aspecto consistente

9. Consideraciones de Seguridad Arquitectónica

Aspecto	Implementación
Contraseñas	bcryptjs con salt=10; nunca almacenadas en texto plano
Autenticación	JWT firmado con HMAC-SHA256; expiración de 1 hora
Autorización	Cada query filtra por <code>user_FK = req.user.user_id</code>
Validación	@hapi/joi valida todo body antes del controller
ORM	Sequelize usa queries parametrizadas (anti SQL Injection)
CORS	Configurado en <code>app.js</code> ; en producción restringir a dominios conocidos
HTTPS	Nginx + Let's Encrypt en producción

10. Glosario Técnico

Término	Definición
API REST	Interfaz de programación basada en HTTP y recursos representados en JSON
JWT	JSON Web Token – token firmado para autenticación stateless
ORM	Object-Relational Mapping – abstracción entre código y base de datos
UUID	Identificador único universal de 36 caracteres (user_id)
FK	Foreign Key – clave foránea que referencia otra tabla
Sequelize	ORM de Node.js compatible con MySQL, PostgreSQL, SQLite, MSSQL
PM2	Process Manager para Node.js; gestiona reinicio y clustering
Nginx	Servidor web y proxy reverso de alto rendimiento
SPA	Single Page Application – la web carga una sola vez y navega sin recargas
Flutter	SDK de Google para apps nativas multiplataforma
bcrypt	Algoritmo de hashing criptográfico para contraseñas